

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0515

Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 10%

QUESTIONS:4

Total Marks 20

Duration :60 Minutes

Annexure A

Mock Interview

Code of Conduct:

I PAVAN JANA (192472313) (Nqme / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: PAVAN J

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.

Answer:

I would design the OpenCV pipeline in the following stages:

Start



Specify folder path



Get all image filenames



Read images using `cv2.imread()`



Check whether image is valid



Resize / Convert / Process image



Display or save result



Process next image



Release resources



End

Explanation:

1. First, I specify the folder containing the images.
2. I use Python file-handling functions such as `os.listdir()` or `glob()` to obtain all image files.

3. Each image is read using `cv2.imread()`.
4. I check whether the image was successfully loaded.
5. Required processing such as resizing, grayscale conversion, filtering, or enhancement is performed.
6. The processed image is displayed using `cv2.imshow()`.
7. A loop is used to process all images automatically.
8. Finally, I close the OpenCV windows using `cv2.destroyAllWindows()`.

Important OpenCV functions:

- `cv2.imread()` – reads an image
- `cv2.resize()` – changes image size
- `cv2.cvtColor()` – changes color space
- `cv2.imshow()` – displays image
- `cv2.imwrite()` – saves image
- `cv2.destroyAllWindows()` – closes windows

Challenges: Large images may consume more memory, corrupted files may fail to load, and processing too many images can reduce performance.

Optimization: Process images one by one instead of loading all images into memory, resize large images, and process only supported image formats.

2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.

Answer:

I would debug the problem step by step rather than immediately changing the code.

Debugging Steps:

Step 1 – Check the file path

Make sure the video path is correct.

```
video =  
cv2.VideoCapture("video.mp4")
```

If the path is incorrect, OpenCV cannot open the video.

Step 2 – Check whether the video opened

```
IF video.isOpened() ==  
    FALSE    Display  "Video  
    cannot be opened"
```

```
END IF
```

Step 3 – Check the video format

The video may use an unsupported codec or format. I would try a common format such as MP4.

Step 4 – Check frame reading

```
ret, frame = video.read()
```

```
IF ret == FALSE
```

```
    Display  "Frame  cannot  be  
read" END IF
```

Step 5 – Check codec/backend

Some videos require a suitable backend or codec. I would verify whether the required codec is available.

Step 6 – Check permissions

I would make sure the program has permission to access the video file.

Step 7 – Test with another video

If another video works, the original video may be corrupted or use an unsupported codec.

Final debugging flow:

Check Path



Check File Exists



Check video.isOpened()



Check ret from video.read()



Check Format / Codec



Test Another Video



Check OpenCV Installation

This structured approach helps identify whether the problem is related to the **path, file, codec, OpenCV, or code**.

3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.

Answer:

Pipeline:

Start Camera



Capture Frame



Process Frame



Draw Text



Draw Shapes



Display Frame



Check Exit Key



Repeat

t

Implementation:

on:

1. Open the camera using `cv2.VideoCapture(0)`.
2. Read each frame using `video.read()`.
3. Use OpenCV drawing functions to add text and shapes.
4. Display the modified frame.
5. Continue until the user presses an exit key.
6. Release the camera and close the window.

Important functions:

- `cv2.putText()` – adds text

- cv2.rectangle() – draws rectangles
- cv2.circle() – draws circles
- cv2.line() – draws lines
- cv2.imshow() – displays the frame
- cv2.waitKey() – handles keyboard

input Example logic:

WHILE camera is

running Capture

frame

Add text using putText()

Draw rectangle using

rectangle() Draw circle using

circle()

Display frame

IF user presses 'q'

BREAK

END IF

END

WHILE

Release

camera Close

windows

Efficiency: I would avoid unnecessary processing, use an appropriate frame resolution, and perform only the required drawing operations on each frame. This helps maintain real-time performance.

4. **An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.**

Answer:

If an image appears too dark, I would first check whether the problem is with the original image, color conversion, or brightness of the image.

Analysis:

Read Image



Check Image



Check Color Space



Analyze Brightness



Apply Brightness/Contrast Enhancement



Display Improved Image

Step 1 – Check image reading

I would verify that `cv2.imread()` has successfully loaded the image.

Step 2 – Check color format

OpenCV reads color images in **BGR format**. If I need to display or process another color space, I would use:

`cv2.cvtColor()`

Step 3 – Increase brightness

I can increase pixel intensity using image addition or suitable brightness adjustment.

Step 4 – Improve contrast

I can use contrast enhancement to make dark and bright regions more distinguishable.

Step 5 – Use histogram-based enhancement

For severely dark images, I can use **Histogram Equalization**
or **CLAHE**. Dark Image



Convert to Grayscale



Apply Histogram Equalization / CLAHE



Enhanced Image

Why this improves visibility: Brightness adjustment makes the image lighter, while contrast enhancement improves the difference between dark and bright regions. CLAHE is especially useful when different parts of the image have different lighting conditions.

Conclusion: I would first identify the reason for the darkness and then select the appropriate enhancement method instead of blindly increasing brightne

